

Singleton – zadania do wykonania:

Zaimplementuj klasę Logger, która będzie pełniła rolę pojedynczego, globalnego obiektu do logowania informacji w aplikacji. Klasa ta powinna być zgodna ze wzorcem Singleton, aby zapewnić dostęp do jednej, wspólnej instancji konfiguracji w całej aplikacji.

Wymagania:

1. Klasa Logger powinna mieć:
 - Prywatny konstruktor.
 - Statyczną właściwość Instance, zwracającą jedyną instancję klasy.
 - Metody:
 - Log(string message), która będzie wyświetlać komunikat na konsoli.
 2. Singleton powinien być bezpieczny między wątkami.
 3. Przetestuj działanie Singleтона w metodzie Main() poprzez wywołanie logowania z różnych miejsc w kodzie.
-

Zaimplementuj klasę ConfigurationManager, która będzie przechowywać ustawienia konfiguracyjne aplikacji w postaci kluczy i wartości. Klasa ta powinna być zgodna ze wzorcem Singleton, aby zapewnić dostęp do jednej, wspólnej instancji konfiguracji w całej aplikacji.

Wymagania:

1. Klasa ConfigurationManager powinna mieć:
 - Prywatny konstruktor.
 - Statyczną właściwość Instance, zwracającą jedyną instancję klasy.
 - Metody:
 - void Set(string key, string value), która zapisuje wartość dla danego klucza.
 - string Get(string key), która pobiera wartość dla klucza (lub zwraca null, jeśli klucz nie istnieje).
2. Klasa powinna przechowywać konfigurację w słowniku (Dictionary<string, string>).

3. Singleton powinien być bezpieczny między wątkami i implementować leniwe ładowanie.
 4. Przetestuj działanie menadżera w metodzie Main().
-

Zaimplementuj klasę CallCounter, która będzie liczyć, ile razy została wywołana jej metoda Increment(). Klasa ta powinna być zgodna ze wzorcem Singleton, aby zapewnić dostęp do jednej, wspólnej instancji konfiguracji w całej aplikacji.

Wymagania:

1. Klasa CallCounter powinna mieć:
 - Prywatny konstruktor.
 - Statyczną właściwość Instance, zwracającą jedyną instancję klasy.
 - Metody:
 - void Increment(), która zwiększa licznik wywołań.
 - int GetCount(), która zwraca aktualną wartość licznika.
2. Przetestuj działanie klasy CallCounter w metodzie Main(), wywołując metodę Increment() kilkakrotnie i sprawdzając wynik.
3. Singleton powinien implementować leniwe ładowanie.